

EXERCISE 3 :

SCENARIO 1:

The bank needs to process monthly interest for all savings accounts.

Write a stored procedure **ProcessMonthlyInterest** that calculates and updates the balance of all **Savings** accounts by applying **1% interest** to the current balance.

QUERY:

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
AS
BEGIN

    UPDATE Accounts

    SET Balance = Balance + (Balance * 1 / 100)

    WHERE AccountType = 'Savings';

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Monthly Interest Applied Successfully.');
```

EXCEPTION

```
    WHEN OTHERS THEN

        ROLLBACK;
```

```
DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;

/

BEGIN

    ProcessMonthlyInterest;

END;

/

SELECT *

FROM Accounts;
```

OUTPUT: BEFORE:

	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE	LASTMODIFIED
1	1	1	Savings	0	6/28/2026, 6:25:43
2	2	2	Checking	2500	6/28/2026, 6:25:43
3	3	3	Savings	12000	6/28/2026, 4:06:41
4	4	4	Checking	18000	6/28/2026, 4:06:41
5	5	5	Savings	9000	6/28/2026, 4:06:41

AFTER:

	ACCOUNTID	CUSTOMERID	ACCOUNTTYPE	BALANCE	LASTMODIFIED
1	1	1	Savings	0	6/28/2026, 6:25:43
2	2	2	Checking	2500	6/28/2026, 6:25:43
3	3	3	Savings	12241.2	6/28/2026, 4:06:41
4	4	4	Checking	18000	6/28/2026, 4:06:41
5	5	5	Savings	9180.9	6/28/2026, 4:06:41

SCENARIO 2:

The bank wants to implement a bonus scheme for employees based on their performance. Write a stored procedure **UpdateEmployeeBonus** that updates the salary of employees in a given department by adding a bonus percentage passed as a parameter.

QUERY:

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
    p_department IN VARCHAR2,
    p_bonus IN NUMBER
)
AS
BEGIN

    UPDATE Employees
    SET Salary = Salary + (Salary * p_bonus / 100)
    WHERE Department = p_department;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Employee Bonus Updated Successfully.');
```

EXCEPTION

```
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;
```

/

OUTPUT:

BEFORE:

	EMPLOYEEID	NAME	POSITION	SALARY	DEPARTMENT	HIREDATE
1	1	Alice Johnson	Manager	102487	HR	6/15/2015, 12:00:00
2	2	Bob Brown	Developer	60000	IT	3/20/2017, 12:00:00

AFTER:

	EMPLOYEEID	NAME	POSITION	SALARY	DEPARTMENT	HIREDATE
1	1	Alice Johnson	Manager	102487	HR	6/15/2015, 12:00:00
2	2	Bob B Alice Johnson	Developer	72600	IT	3/20/2017, 12:00:00

SCENARIO 3:

Customers should be able to transfer funds between their accounts. Write a stored procedure **TransferFunds** that transfers a specified amount from one account to another, checking that the source account has sufficient balance before making the transfer.

QUERY:

```
CREATE OR REPLACE PROCEDURE TransferFunds(
    p_from_account IN NUMBER,
    p_to_account IN NUMBER,
    p_amount IN NUMBER
)
AS
    v_balance Accounts.Balance%TYPE;
BEGIN

    -- Get source account balance
    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = p_from_account;

    -- Check sufficient balance
    IF v_balance >= p_amount THEN

        -- Deduct amount from source account
        UPDATE Accounts
        SET Balance = Balance - p_amount
        WHERE AccountID = p_from_account;

        -- Add amount to destination account
        UPDATE Accounts
        SET Balance = Balance + p_amount
        WHERE AccountID = p_to_account;

        COMMIT;

        DBMS_OUTPUT.PUT_LINE('Transfer Successful');

    ELSE

        DBMS_OUTPUT.PUT_LINE('Insufficient Balance');

    END IF;

EXCEPTION

    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Invalid Account ID');

    WHEN OTHERS THEN
```

```

ROLLBACK;
DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);

END;
/

```

OUTPUT: BEFORE:

	ACCOUNTID	BALANCE
1	1	0
2	2	2500
3	3	12241.2

AFTER:

```

BEGIN
    TransferFunds(1,2,500);
END;
/

```

	ACCOUNTID	BALANCE
1	1	0
2	2	2500
3	3	12241.2